

Curve: Voor het definiëren van de af te leggen route door de ruimte maakt de applicatie gebruik van Bézier-curves. Om een complexer en langer parcours te vormen, is de volledige route opgebouwd uit twee afzonderlijke curves die aan elkaar zijn gekoppeld. De berekening van de punten is geoptimaliseerd door het toepassen van de forward differencing methode. Hierbij wordt enkel de wiskundige verandering (delta) per stap berekend en iteratief opgeteld, wat rekenkracht bespaart ten opzichte van de brute-force methode. Elke curve is geconfigureerd om te worden opgedeeld in exact 50 stappen. Dit resulteert in een gecombineerd, doorlopend pad van 100 segmenten waarvan de resulterende ankerpunten worden opgeslagen in de lijst `fullRockPath`.

Animatie: Om te zorgen dat een object met een constante snelheid over het Bézier-pad beweegt, wordt gebruikgemaakt van een look-up table. Tijdens de animatie berekent de code hoe ver het schip al moet zijn. Daarna wordt via de tabel de juiste t-waarde gevonden en daarmee de positie op de curve berekend. Zo beweegt het schip gelijkmatig over het pad en kijkt het mee in de juiste richting.

Modellen & Textures: Voor de 3D-modellen (zoals de TIE-fighter, Star Destroyer en stenen) en textures hebben we gratis bestanden van het internet gebruikt. De `.gltf`- en `.obj`-modellen worden ingeladen met Assimp, terwijl de `.png`, `.jpg` en `.jpeg` textures van deze modellen worden ingeladen met `stb_image`. Het uiteindelijke tekenen naar het scherm doen we door het te binden aan de ingeladen vertex- en texturedata handmatig aan Vertex Array Objects.

Visualisatie: Om de berekende curve zichtbaar te maken in de applicatie en het traject visueel aan te kleden, wordt de data uit `fullRockPath` gebruikt voor de rendering van een organisch pad. Voor elk van de 100 segmenten worden 12 exemplaren van een in Blender ontworpen 3D-stenenmodel gerenderd, wat leidt tot een totaal van 1200 stenen.

Om de objecten vloeiend te laten meebuigen met het traject, wordt per segment de lokale curve-richting (de raaklijn of tangent) berekend. Deze richting wordt via geometrische transformaties gebruikt voor de correcte baangeleiding en rotatie van de stenen. Om het statische "puzzelstukjes"-effect te doorbreken en een natuurlijke look te creëren, is er een willekeurige jitter (variëaties in positie, schaal en rotatie) op de transformatiematrices van de stenen toegepast.

Camera: We hebben een duaal camerasysteem geïmplementeerd waarmee de gebruiker via de T-toets kan wisselen tussen twee manieren. De vrije camera gebruikt een klassieke LookAt-matrix waarmee je met de muis en de W, A, S, D-toetsen volledig vrij door de scène kunt navigeren om het parcours te bekijken. Deze is volledig gebaseerd op de learnopengl course content. De first-person camera koppelt de

camerapositie en de kijkrichting direct aan de actuele positie en de bewegingsvector van de Tie Fighter op de Bezier-curve, waardoor je de vlucht nauwkeurig vanuit de cockpit kunt meevolgen.

Belichting: Voor de belichting hebben we het Phong verlichtingsmodel geïmplementeerd in de fragment shader. De zon fungeert hierin als onze lichtbron die ambient, diffuse en specular licht werpt op Saturnus, het stenen pad en de ruimteschepen. Om te voldoen aan de eis van attenuation zorgen we ervoor dat op basis van de afstand tussen de lichtbron en het object de intensiteit van het licht toe- of afneemt naarmate objecten verder of dichterbij de bron staan.

Convolutie: We implementeren post-processing door de scène eerst te renderen naar een framebuffer, die is uitgewerkt in een aparte klasse. Vervolgens wordt de inhoud van deze framebuffer als textuur op een screen aligned quad getekend wat dus een rechthoek is die het volledige scherm vult.

Tijdens het tekenen van deze quad passen we convolutie toe via een aparte shader. Afhankelijk van de gekozen modus wordt een specifieke kernel gebruikt: een Gaussian blur kernel voor vervaging of een Laplacian edge kernel voor randdetectie. De sterkte van de blur is verhoogd voor een duidelijker effect. De gebruiker kan live wisselen tussen de effecten: Gaussian blur met toets 1, Laplacian edge detectie met toets 2, en terug naar het standaardbeeld met toets 0.

Post-processing (Bloom-effect): Voor post-processing hebben we een bloom-effect geïmplementeerd. Hiervoor worden de lichtbronnen, zoals de zon, eerst geïsoleerd en apart gerenderd op een zwarte achtergrond. Vervolgens wordt op deze afbeelding een Gaussian blur toegepast, waardoor de lichtbronnen een zachte gloed krijgen. Ten slotte wordt het resultaat van deze geblurde lichtbronnen gecombineerd met de originele scène. Hierdoor ontstaat een zichtbaar bloom-effect rond de lichtbronnen, terwijl de rest van de scène scherp blijft. Zo lijken de lichtbronnen extra fel te schijnen zonder dat de hele scène wazig wordt.

Chroma-Keying: We gebruiken een greenscreen-texture met een helm als overlay. Ook hier maken we gebruik van een screen aligned quad en een framebuffer om het effect zichtbaar te maken. In de fragment shader wordt voor elke pixel gecontroleerd of deze binnen een bepaalde drempelwaarde van groen valt. Is dat het geval, dan wordt de pixel met discard weggegooid en dus volledig transparant gemaakt. Hierdoor blijft alleen het niet-groene deel van de overlay zichtbaar, zoals de helmrand, en zie je de 3D-scène erachter.

Om dit effect te activeren, moet je eerst op 'T' drukken om in de ship camera te komen, en daarna op 'H' (van helm) om de overlay te tonen.

Interactie: De interactie via picking is geïmplementeerd met color picking. Wanneer de gebruiker met de linkermuisknop op de Naboo Fighter klikt, wordt dit object tijdelijk met een unieke kleur gerenderd. De code leest daarna de kleur van de pixel onder de muis uit en zet deze terug om naar een ID. Als deze ID overeenkomt met de Naboo Fighter, wordt de laser van de TIE Fighter geactiveerd. De laser vertrekt vanuit de huidige positie en oriëntatie van de TIE Fighter en verdwijnt opnieuw nadat hij een bepaalde afstand heeft afgelegd.

Planning

07/04	Eerste volledige projectdag; beiden aan de hoofdstukken van LearnOpenGL gewerkt.
07/04 - 16/04	Individueel verder gewerkt en geoefend met de LearnOpenGL-tutorial.
16/04	Samen gedurende enkele uren het initiële project opgezet en werkend gekregen in Live Share, op basis van LearnOpenGL-code en -kennis.
Elke dinsdag van 21/04 - 12/05	Quasi chronologisch de deeltaken uit de projectopdracht afgehandeld.
Meerdere weekenden van 25/04 - 16/05	Individueel toegewezen taken die nog openstonden van de dinsdagen afwerken.
18/05	Verlag + video afgerond.